Matriz de atendimento - Fase 3

Revisão: 31/10/2026 (BRT). Conferência baseada no enunciado fornecido, no código, nos workflows e em testes pelo API Gateway. Esta matriz registra a validação anterior ao encerramento do Learner Lab solicitado pelo responsável.

Requisitos e evidências

Requisito	Resultado e evidência
API Gateway e rotas protegidas	HTTPS, integração privada, Swagger/OpenAPI e Authorizer validados no deploy completo
Autenticação serverless por CPF	CPF válido de CUSTOMER ativo retorna 200/JWT; CPF inválido retorna 400; cliente sintético inativo retorna 401 no teste público final. O cliente exclusivo de validação foi removido após o teste
JWT e RBAC	CUSTOMER consulta suas OS com 200; ausência de token 401; token malformado/alterado e acesso operacional indevido 403. Matriz do Authorizer e smoke do deploy atual
Ciclo de OS	As 12 requisições da coleção passaram: abertura 201, diagnóstico 200, orçamento 201, login CPF 200, consulta 200, erros 401/403, aprovação 200, conclusão 200, entrega 200 e leitura final DELIVERED 200
Quatro repositórios e proteção de main	Aplicação, Lambda, Kubernetes e banco separados; rulesets exigem PR; READMEs contêm propósito, stack, diagramas, execução/deploy e links da API
CI/CD e deploy por branch	Workflows validam PRs e publicam automaticamente pushes em homolog/production. Execução real foi comprovada em homolog; não se declara execução de produção neste Lab
RDS PostgreSQL gerenciado	Consumido pela aplicação e pela autenticação, com cliente ativo/inativo verificado pelo fluxo público. Apply do banco
Kubernetes escalável	EKS, Metrics Server e HPA 1-6 pods. Teste aprovado: 704 tentativas de carga, zero erros, 2 -> 5 pods Ready -> 2. Revalidação após os indicadores de negócio: 654 tentativas, zero erros, 2 -> 5 -> 2 pods, p95 cliente 443ms
Terraform e containers	Estados separados, planos com lock, EKS/Gateway/RDS/Lambdas provisionados; Dockerfile da aplicação. Lambdas usam ZIP Go; os repositórios Terraform não exigem container de runtime
CPU, memória e health/uptime	Dois dashboards FIAP; memória em MB, CPU em mCPU, probes públicas e readiness/liveness. Histórico depende da janela e da vida dos pods da stack
Latência, tráfego e erros HTTP	Histogramas e contadores publicados após corrigir limites de Timer para nanossegundos; p95 e taxa por rota consultáveis. Sem tráfego recente, p95 não possui amostra; não se substitui por um zero fictício
Volume diário de OS	Gauge persistido `workshop_work_orders_opened_today`, consultado do PostgreSQL a cada 30s; painel usa max entre réplicas. Dia do banco em UTC; não soma o mesmo agregado por pod
Duração média por status	`workshop_work_order_stage_duration_seconds`: DIAGNOSIS (Diagnóstico), IN_PROGRESS (Execução), COMPLETED até DELIVERED (Finalização). Considera etapas encerradas do histórico e exclui OS canceladas; sem etapa encerrada, mantém ausência de amostra
Falhas de integrações	Contadores dedicados de sucesso/falha para SMTP e coleta dos indicadores no PostgreSQL, com painel por integração. O SMTP do sandbox apresentou falhas reais; a OS continuou normalmente e o sinal foi registrado
Alertas de OS	Regras `WorkOrderProcessingFailure` (5xx em rota de OS) e `WorkOrderNotificationFailure` (SMTP). 4xx esperados não disparam falha de processamento. Teste promtool comprova firing/resolved; auditoria observou alerta SMTP real em firing

Requisito	Resultado e evidência
Logs e traces correlacionados	JSON/MDC com requestId e traceId; log de conclusão sem CPF, token, senha ou corpo. Auditoria encontrou traces reais da aplicação no Tempo e logs com os mesmos identificadores, sem exportar seus valores
Arquitetura e modelo de dados	Componentes, sequências de CPF/abertura de OS, RFCs, ADRs, ER e justificativa formal do PostgreSQL publicados neste diretório, com PDFs e diagramas separados
Vídeo <=15 minutos	Demonstração Fase 3, 13min29s. Acesso/duração conferidos nos metadados; o autor confirmou a demonstração de logs e traces. Não se apresenta essa confirmação como revisão independente do vídeo
PDF e avaliador	PDF único do Portal com quatro repositórios, vídeo, documentação e confirmação de `soat-architecture`, reconfirmado via API nos quatro repositórios

Resultados HTTP do teste público final e resumo persistido da carga/HPA final, sem identificadores ou credenciais.

Evidência final da observabilidade

Auditoria com verificações obrigatórias: volume diário disponível, três séries de duração, contadores de integração/processamento, duas regras de OS e traces com logs correspondentes. Resultado: `phase3_observability_validation PASSED`.

A auditoria observou `WorkOrderNotificationFailure=firing` após falhas SMTP reais; a conferência posterior confirmou a resolução para `inactive` e passou novamente em todas as verificações. `WorkOrderProcessingFailure` permaneceu inativo durante o ciclo da OS bem-sucedido. As regras e o teste reprodutível de disparo/resolução estão em observability/rules e observability/tests. Alertas são consultáveis no Grafana/Prometheus/Alertmanager; não foi configurada entrega externa por e-mail ou chat.

Validação de código

- Aplicação: `./mvnw verify` completo em Java 21, inclusive após consolidar homolog em main;

PR 90 com CI aprovado.

- PostgreSQL 16: consulta dos agregados verificada com datas conhecidas, três durações,

abertura diária e exclusão de cancelamentos/etapas ainda abertas.

- Lambda: testes Go e `go vet`, além dos checks do PR de consolidação.

- Terraform: formatação/validação de PRs e planos de remoção limitados aos recursos FIAP.

Limites conhecidos do ambiente acadêmico

- Observabilidade usa emptyDir: histórico de métricas/logs/traces é volátil. EBS CSI

não foi novamente aplicado após os bloqueios IAM do Academy.

- Os 503 da carga antiga não foram reproduzidos no teste aprovado de 704 tentativas;

isso comprova aquela janela e carga limitada, sem afirmar capacidade ilimitada ou uma causa única.

- O SMTP é sandbox e pode recusar entregas por limite/indisponibilidade. Essas falhas

são sinalizadas e não revertem o atendimento; o teste do ciclo não promete entrega de e-mail.

- Homolog foi executado; production possui automação configurada e não foi implantado

apenas para produzir evidência adicional ou aumentar custo no Lab.

- Os endpoints de runtime dependem dos recursos AWS. Após o destroy solicitado,

a avaliação deve usar o vídeo, código, diagramas e evidências dos workflows.

Encerramento após a validação

Registro completo. Todas as stacks foram removidas, com zero recursos gerenciados, e a verificação final com limpeza do bucket passou. Os endpoints de runtime e a massa deixaram de existir após o destroy solicitado; vídeo e repositórios permanecem disponíveis.